

Human vs machine

A human and a machine will solve this maze in similar yet distinct ways. A human will use a combination of visual senses, memory, and problem solving. In the game, the human player is tasked with finding the treasure before the AI pirate does. When starting the game, the player will immediately notice that the path to the right is blocked; there is only one way to go, which is down. When they move down again, they see the only move is down because the right path is blocked. A human is able to remember these paths are blocked and store them in their memory. They can navigate around the maze and explore until they find the treasure (Zhao, 2014).

A machine or the pirate AI uses a similar approach, but the mechanisms in how it knows where to go are different. The pirate will rely on trial and error much more than the human player. Starting the game, the pirate cannot see that the only valid first two moves are down. It may try to go up, left, or right, which would put it out of the play area or into the two blocked squares. Using reinforcement learning, the pirate can learn it cannot move to those squares. The pirate will explore all cells until it reaches the treasure through pure trial and error. As the pirate plays the game more, however, it will become increasingly better as it is able to exploit the paths it has already found. In longer and more complex mazes, the pirate has the advantage of reusing where it has already gone, as a human may struggle to memorize their previous paths. The human has the initial advantage because they do not need to explore paths they know will not work, such as moving into the invalid squares at the start of the game (Baker et al., 2019). The machine can never see the maze like a human does; it learns through states and actions.

Intelligent agent in pathfinding

Two very important concepts that aid the pirate in his search for the treasure are exploitation and exploration. Exploitation is using what the pirate has already explored to maximize its rewards. For example, the agent making any other move than down in the starting position is invalid. After making these moves, the pirate learns those are the only moves it can make, so it is exploiting what it has learned. Exploration is the process of exploring new actions or squares to obtain more information about its environment (GeeksforGeeks, 2025; Lamba, 2018). When the pirate starts the game, it does not know where it can go, but it must move somewhere, so it goes up. Moving up is outside the play area, and the pirate is penalized for making an invalid move.

In reinforcement and pathfinding problems, it is particularly important to balance these. If we have a pirate that only exploits or chooses the highest reward now, it will not explore the squares it needs to find the treasure. If the pirate explores too much, then it will take random actions to learn about its environment but will never learn how to exploit it. This will result in low rewards because the pirate is moving around the maze randomly (Gulli & Pal, 2017; GeeksforGeeks, 2025). For the pirate to find the treasure, our exploration/exploitation ratio is not a fixed number for the problem. We start at 100% exploration, and for each episode or time the pirate goes through the maze, we decrease it by about half a percent. Over time, this allows the pirate to explore new actions and learn from its environment, especially at the start of the game. As the pirate plays more games, however, it starts to learn where it can and cannot go, as well as a path to the treasure. Once the pirate has a promising idea of how to get to the treasure, the exploration factor will be lower, and the pirate will exploit the best path to the treasure every single time. If we kept exploration too high, then the agent would not be able to find the optimal

path every single time. And if we kept exploration too low, the agent would struggle to find it in the first place (Gulli & Pal, 2017).

Reinforcement learning is a key part of how the pirate navigates the maze. Without it, the pirate cannot determine between a valid or invalid move. We mentioned above that the only first two moves the pirate can make at the start are down. If the pirate makes any other move, it is penalized, and the algorithm will learn to only go down because they are the only valid moves. Over time, as the pirate explores more of the maze and the exploration factor is decreased, the pirate can leverage the reward system to slowly find the best path to the treasure (Lamba, 2018). The ethical implications of the pirate agent center around these rewards. What the agent counts as a win or invalid move is dependent on how we reward it. This is why reward design needs to match what we want the agent to do.

AI to solve complex problems

The pirate in this game is using a deep-Q-learning algorithm using neural networks. In a standard Q-learning algorithm, a table called the Q-table will be used to store every outcome or square in the maze. The problem with this approach is that in complex environments like a larger maze, the Q-table cannot store every outcome. Deep-Q-learning replaces the Q-table with a neural network that predicts rewards based on the actions the pirate has taken. The neural network not only allows the agent to navigate larger and more complex environments, but it also lets it make decisions in similar grid environments it has not seen before (Gulli & Pal, 2017; Serengeti, 2019).

In this pirate game, the pirate can take four actions: left, right, up, or down. The state we input to our neural network is the maze or an array stating where the walls are and where the

pirate/treasure is. We created a model that takes in the maze or array and passes it through two hidden layers plus the input/output layer. The network will output four neurons, each representing a possible move (up, down, left, right). The neuron with the highest number is the “best” move (Hsu, 2020). Let us say at the start of the game, the pirate moves up and left, which takes it out of the play area, leading to it losing reward points. The neural network in the next games will output that up and left have a negative reward, so we should go down or to the right.

For the network to remember what it has done, we are using experience replay. This allows the pirate to save a state of the maze that it can pull from later. Experience replay not only allows the network to store things but helps with overfitting and learning because when each experience or state of the maze is pulled, it is done so randomly. Lastly, using the qtrain function, we put everything together. The agent will explore the maze using the explore/exploit ratio and the neural network. The pirate will be given a reward based on its moves, and the results are stored in the experience replay memory for later use. After the network gets a few states in experience replay, the neural network is updated, and it will output better Q-values.

References

- Baker, B., Kanitscheider, I., Markov, T., Wu, Y., Powell, G., McGrew, B., & Mordatch, I. (2019, September 17). Emergent tool use from multi-agent interaction. OpenAI.
<https://openai.com/index/emergent-tool-use/>
- GeeksforGeeks. (2025, July 23). Exploitation and exploration in machine learning.
<https://www.geeksforgeeks.org/machine-learning/exploitation-and-exploration-in-machine-learning/>
- Gulli, A., & Pal, S. (2017). Deep learning with keras : Implementing deep learning models and neural networks with the power of python. Packt Publishing, Limited.
- Hsu, H. (2020, August 5). How do neural network systems work? Computer History Museum.
<https://computerhistory.org/blog/how-do-neural-network-systems-work/>
- Lamba, A. (2018, August 27). A brief introduction to reinforcement learning. Medium.
<https://medium.com/free-code-camp/a-brief-introduction-to-reinforcement-learning-7799af5840db>
- Serengeti. (2019, June 18). Using Q-learning for pathfinding.
<https://serengetitech.com/tech/using-q-learning-for-pathfinding/>
- Zhao, M. (2014). Understanding humans' strategies in maze solving from eye-hand coordination (Doctoral dissertation, Rutgers University). RUcore: Rutgers University Community Repository. <https://doi.org/10.7282/T38W3FX5>